

Participant Handbook

Five tracks. Fifteen build challenges. One shared standard for rigorous, safe, measurable agentic systems.

Build systems that reason, act, pay, defend, coordinate, and improve — with evidence that they work.

Track	Focus
1 Agentic Ecosystem	Coding harnesses, research agents, and multi-agent orchestration
2 Cybersecurity	Forensics, authorized security validation, and unified cyber defense
3 SDG	Civic redressal, disaster response, and mission operations
4 Web3 Agentic Payment Applications	Standard x402-powered pay-per-use services
5 Web3 Agentic Payment Infrastructure	Composite routing, settlement, and orchestration layers

Participant edition

Use this handbook to choose a track, scope a credible prototype, and prepare a submission that can survive technical, safety, and reproducibility review.

Contents

The handbook begins with the shared evaluation contract and then moves through five tracks. Every problem statement uses the same pattern so that participants can compare scope, evidence, and risk consistently.

Section	What it contains
Shared evaluation framework	How every project is judged
Track 1 Agentic Ecosystem	Three problem statements
AE-01 Unified Agentic Coding Harness	Memory, context, tools, and benchmark performance
AE-02 Self-Evolving Autonomous Research Agent	Live RAG, browser use, sandboxing, and governed self-improvement
AE-03 Unified Agent Form Orchestrator	Visual agent design, sequential/parallel execution, and model routing
Track 2 Cybersecurity	Three problem statements
CY-01 Digital Fraud & Synthetic-Media Forensics	Privacy-preserving detection and evidence
CY-02 Authorized Agentic Penetration-Testing Harness	User-owned targets in isolated labs
CY-03 Unified AI Cyber Defense Platform	Phishing, DDoS, and network anomaly detection
Track 3 SDG	Three problem statements
SDG-01 Community Redressal Planner	From complaint intake to accountable resolution
SDG-02 Disaster Management System	Satellite mapping and natural-calamity detection
SDG-03 Space Mission Operations Automator	Simulation-first mission planning and telemetry
Track 4 Web3 Agentic Payment Applications	Three standard x402 application challenges
PAY-01 Pay-per-Use AI & Data API Marketplace	Autonomous payment for metered resources
PAY-02 Agentic Research and Content Paywall	Evidence-linked access to premium information
PAY-03 Machine-to-Machine Compute and Tool Exchange	Agents buying compute and tools on demand
Track 5 Web3 Agentic Payment Infrastructure	Three composite/orchestrator challenges
INF-01 Multi-Provider Agent Payment Router & Treasury	Route, budget, reconcile, and recover
INF-02 Facilitator and Settlement Mesh	Verification, multi-network settlement, and receipts
INF-03 Composite Agentic Commerce Orchestrator	Multi-step workflows with conditional payment
Submission rules	Required package, evidence, and demo expectations
Safety boundaries	Non-negotiable limits for participant systems
Glossary and protocol notes	Shared terms, including x402

Shared evaluation framework

All fifteen problem statements share one evaluation contract. A strong submission is not only a working demo: it is a bounded system with a clear threat model, reproducible tests, visible evidence, and an honest account of failure modes.

The common bar

1. Build a complete vertical slice rather than a collection of disconnected screens or notebooks.
2. Separate model capability from harness, orchestration, memory, tool, and data contributions.
3. Use public datasets, synthetic environments, or organizer-provided tasks with documented licenses and provenance.
4. Report both successes and failures, including false positives, unsafe actions prevented, latency, cost, and human interventions.
5. Design for replay: a judge should be able to run the same scenario again and inspect the trace.
6. Make the system safe by construction; do not rely on a final disclaimer to compensate for unrestricted capabilities.

Judging criteria

Criterion	Weight	What judges look for
Technical depth	25%	Quality of architecture, data flow, model/tool boundaries, and implementation choices.
Verified usefulness	20%	A clear user problem, credible workflow, and measurable value beyond a generic chat interface.
Reliability and evaluation	20%	Repeatable tests, hidden or held-out evaluation, failure analysis, and honest metrics.
Safety and governance	15%	Least privilege, authorization, privacy, approval gates, auditability, and rollback.
User experience	10%	Clarity of setup, interaction, outputs, explainability, and recovery from errors.
Originality and leverage	10%	A distinctive insight, useful composition, or system improvement that generalizes.

Evidence expected in every submission

Evidence	Minimum expectation
Architecture	One-page system diagram plus a short explanation of control plane, data plane, model calls, storage, and external dependencies.
Evaluation	A reproducible runner or script, task list, baseline, metrics, and result table. Include at least one failure case.
Traceability	A run ID or exportable trace showing inputs, decisions, tool calls, outputs, and approval points.
Safety	Threat model, permission model, data handling notes, and a demonstration of a blocked or approval-gated action.
Demo	A five-to-eight-minute walkthrough using a prepared scenario and one unseen or held-out scenario where practical.

How to scope the build

7. Choose one primary workflow and make it excellent before adding breadth.
8. Use deterministic adapters, mocks, and testnets where a live dependency would create unnecessary risk or cost.
9. If your project claims autonomy, define the exact actions it may take without approval and the exact conditions that pause it.
10. If your project claims learning or self-improvement, show a before/after evaluation on held-out tasks; a changed prompt is not evidence of improvement by itself.

TRACK 1

Agentic Ecosystem

Build the control plane around intelligence.

This track rewards systems that improve existing models through memory, context engineering, tools, planning, verification, and disciplined orchestration.

Participants may build one of the three challenges below. The most competitive entries isolate the contribution of the harness from the raw capability of the underlying model and show the difference with repeatable measurements.

AE-01

Unified Agentic Coding Harness

Problem statement Design a model-independent terminal coding harness that can understand, modify, test, debug, review, and maintain real repositories while proving that its harness architecture improves performance under fixed model and budget conditions.

AT A GLANCE

Design dimension	Participant expectation
Primary user	Developers and engineering teams working across large, polyglot repositories.
Core inputs	Issue, repository, constraints, tools, tests, history, and user-approved project memory.
Required outputs	Verified patch, trace, test evidence, cost/latency report, and rollback-ready artifact.
Hardest challenge	Separating model capability from context, memory, orchestration, and verification gains.
Proof of quality	Terminal-Bench/SWE-bench-style results plus ablations and hidden tasks.

Core objective

Build a production-minded coding agent that can receive a high-level software-engineering request and complete the loop from repository onboarding through evidence-backed patch verification. Study useful ideas from Claude Code, OpenCode, Amp, Droid, Codex CLI, and Pi, but produce an original system rather than a feature checklist or clone.

Required capabilities and system blueprint

- Model-independent adapter layer separating model calls from context construction, memory retrieval, tool execution, policy, planning, verification, and instrumentation.
- Repository intelligence combining file maps, AST/symbol indexing, import and call graphs, test-to-source mapping, configuration relationships, and git history.
- Tiered memory for working state, task state, project conventions, episodic outcomes, reusable procedures, user preferences, and verified evidence. Every memory item needs provenance and invalidation rules.
- Context manager with dynamic token budgets, relevance scoring, hierarchical summaries, stale-context detection, prompt-injection resistance, and refresh after code changes.
- Task graph with sequential and parallel branches, bounded specialist agents, independent review, checkpoints, early termination, and dynamic replanning.
- Sandboxed command execution with scoped filesystem, network policy, resource limits, secret isolation, snapshots, approval gates, and emergency termination.
- Verification-first completion using builds, tests, linters, type checks, static analysis, regression tests, and hidden evaluator tasks rather than model confidence.
- Observability that shows plan revisions, retrieved context, tool calls, files touched, tests, failures, recovery actions, token/cost usage, and final evidence without exposing hidden chain-of-thought.

Minimum viable demonstration

- Repository intake creates an initial map and asks for a bounded issue or feature request.
- The harness reproduces the issue, records an evidence item, and produces an explicit task graph.
- A coding run produces a patch, runs the declared verification suite, and recovers from at least one injected failure.
- A comparison run shows the same model with a baseline harness versus the submitted harness under the same budget.
- A reviewer view explains why the patch is complete, what remains uncertain, and how to roll it back.

Evaluation and benchmarks

- Terminal-Bench as the primary task family; SWE-bench Verified/Lite, LiveCodeBench, repository-understanding tasks, and organizer-provided hidden issues as complements.
- Report pass rate, hidden-test pass rate, cost per successful task, tokens, tool-call failures, recovery rate, stale-context rate, human interventions, and safety-policy violations.
- Run ablations: memory on/off, structural retrieval on/off, single-agent/multi-agent, cold/warm memory, and baseline/submitted harness with the same model.

Hard-mode extensions

27. Large monorepositories, sparse documentation, polyglot services, flaky tests, generated code, dependency migrations, and tasks requiring more than one hour of work.
28. Model replacement without harness changes, restricted network, hidden tests, contradictory requirements, and deliberately stale project memory.

Safety boundary Only operate on repositories and credentials the participant controls. The harness must never receive unrestricted host credentials, private files, SSH keys, browser sessions, or production deployment authority. No benchmark gaming through leaked tests or hidden-task inspection.

Expected deliverables

29. Working CLI or terminal UI and model/tool adapter SDK.
30. Context and memory engine with inspect/edit/export/delete controls.
31. Sandboxed execution service, trace viewer, and reproducible benchmark runner.
32. Architecture, threat-model, ablation, failure-analysis, and cost report.
33. Demonstration on an unseen repository with a rollback-ready patch.

Why this stands out

This problem isolates a high-leverage systems question: how much more can existing models accomplish when the harness treats context, memory, tools, and verification as first-class engineering surfaces? It is difficult to fake because the contribution must survive controlled comparisons, not just a polished demo.

AE-02

Self-Evolving Autonomous Research Agent

Problem statement Build a secure research and computer-use agent that can browse, retrieve evidence, execute code, operate an isolated workspace, and improve its strategies over time without silently rewriting its trusted control layer.

AT A GLANCE

Design dimension	Participant expectation
Primary user	Researchers, analysts, and teams running multi-step investigations.
Core inputs	Question, source policies, browser/tool capabilities, documents, datasets, and prior project memory.
Required outputs	Evidence-backed report, citations, reproducible artifacts, trace, and improvement log.
Hardest challenge	Making self-improvement measurable while preventing unsafe drift and prompt-injection takeover.
Proof of quality	Longitudinal success curve, citation metrics, injection resistance, and rollback tests.

Core objective

Create an agent that turns an open-ended question into a reproducible research package through live retrieval, document analysis, code execution, browser navigation, and controlled computer interaction. The defining feature is governed self-evolution: the agent may propose better prompts, tool-selection policies, retrieval strategies, or reusable skills, but each change must be evaluated and approved before it becomes trusted behavior.

Required capabilities and system blueprint

34. Research planner producing a graph of subquestions, evidence requirements, source classes, confidence thresholds, and stop conditions.
35. Hybrid live RAG with keyword and dense retrieval, time-aware ranking, source freshness, deduplication, contradiction tracking, and provenance for every passage.
36. Browser and file controller that treats webpages, PDFs, comments, advertisements, and downloads as untrusted input and defends against prompt injection.
37. Sandboxed notebook/terminal workspace with disposable sessions, snapshots, resource quotas, network allowlists, download quarantine, and audit logs.
38. Evidence graph linking claims to sources, timestamps, locations, transformations, and uncertainty; outputs must distinguish observation from interpretation.
39. Controlled self-improvement loop that logs strategy outcomes, critiques failures, proposes a versioned change, tests it on held-out tasks, and supports rollback.
40. Human approval gates for sensitive actions such as external account changes, publication, communication, deletion, payments, or access to private data.
41. Long-term project memory for source summaries, contradictions, research methods, user preferences, and unresolved questions with expiry rules.

Minimum viable demonstration

42. Give the agent a complex question requiring at least three source types, one structured dataset, and one code-based calculation.
43. Inject a malicious instruction into a webpage or downloaded document and show that the agent ignores it while preserving useful evidence.
44. Let the agent propose a retrieval or tool-selection improvement, run it against held-out tasks, and compare old versus new strategy.
45. Produce a cited report, a source/evidence map, a reproducible data artifact, and an exportable action trace.

Evaluation and benchmarks

46. GAIA, WebArena/VisualWebArena, OSWorld, PaperBench-style replication, citation validation, long-context synthesis, and organizer-provided live-information tasks.
47. Measure end-to-end success, citation precision/recall, unsupported-claim rate, source quality, browser success, code-execution success, prompt-injection resistance, cost, and human interventions.

48. Add a longitudinal benchmark across multiple sessions to show whether the self-evolution loop improves held-out performance rather than merely changing behavior.

Hard-mode extensions

49. Contradictory sources, multilingual material, malformed scans, dynamic websites, large document collections, stale evidence, and tasks requiring browser plus terminal plus desktop tools.
50. A self-proposed skill that attempts to expand its permissions, alter a policy file, or reach an unapproved network must be blocked and reported.

Safety boundary System control, scraping, and self-evolution are permitted only inside a disposable, capability-scoped workspace. Do not bypass CAPTCHAs, access private accounts without approval, exfiltrate credentials, or let generated skills modify trusted control code without review and rollback.

Expected deliverables

51. Research planner, retrieval/indexing pipeline, browser/desktop adapters, and sandbox service.
52. Evidence/citation graph, memory store, self-improvement evaluator, policy engine, and trace viewer.
53. Longitudinal benchmark with before/after strategy results and held-out tasks.
54. Threat model, prompt-injection tests, data-handling notes, and rollback demonstration.
55. Cited final report plus machine-readable research package.

Why this stands out

Most research agents stop at retrieval and summarization. This challenge asks whether an agent can learn better ways to research while remaining inspectable, reversible, and safe — a much harder claim than adding another search box to a chatbot.

AE-03

Unified Agent Form Orchestrator

Problem statement Create a single app that lets users configure a commercial or local LLM, describe a goal, generate a typed network of specialized agents, and execute that network sequentially or in parallel with visible state, budgets, permissions, and verification.

AT A GLANCE

Design dimension	Participant expectation
Primary user	Teams that need repeatable multi-agent workflows without hand-coding every agent.
Core inputs	Goal, model/provider, agent roles, tools, memory policy, data, and approval rules.
Required outputs	Runnable workflow, per-agent artifacts, merged result, trace, cost, and replayable configuration.
Hardest challenge	Turning natural-language task decomposition into reliable, typed, observable execution.
Proof of quality	Workflow success, handoff correctness, recovery, cost controls, and reproducible replays.

Core objective

Build a unified orchestration surface — visual, CLI, or API-first — where a user can bring an LLM key or local model, define a task, and generate or configure a set of agents that collaborate. The system must support sequential pipelines, parallel fan-out/fan-in, conditional branches, shared or isolated memory, typed handoffs, approval points, and independent verification.

Required capabilities and system blueprint

56. Provider abstraction for commercial and local models with per-agent model selection, fallback, streaming, rate-limit handling, and secrets isolation.
57. Agent templates with role, system contract, input/output schema, tools, memory scope, budget, timeout, and allowed side effects.
58. Task-to-graph compiler that proposes a workflow and lets a user inspect, edit, lock, version, and approve the graph before execution.
59. Execution engine for sequential, parallel, conditional, retry, join, human-review, and compensation nodes with deterministic state transitions.
60. Typed message bus and artifact store so agents exchange validated outputs rather than dumping unstructured transcripts into one context.
61. Shared project memory plus per-agent scratch memory with provenance, access control, retention, and conflict resolution.
62. Observability dashboard showing live graph state, token/cost budgets, tool calls, latency, retries, failures, and evidence for completion.
63. Evaluation harness that compares single-agent, manually designed multi-agent, and generated workflows on the same tasks.

Minimum viable demonstration

64. Enter a complex goal and produce a proposed graph with at least one parallel branch and one verification node.
65. Run the graph on a fixed dataset or repository, show typed handoffs, and intentionally fail one branch to demonstrate retry or compensation.
66. Change the model used by one agent or rerun the same graph with a local model without rewriting the workflow.
67. Replay a completed run from saved configuration and compare cost, latency, and outputs against the original.

Evaluation and benchmarks

68. BFCL-style tool calling, AgentBench-style tasks, graph-execution reliability tests, structured-output validation, and organizer-provided long-horizon workflows.
69. Measure task success, handoff validity, branch utilization, recovery rate, duplicate work, cost per successful run, latency, approval frequency, and reproducibility.
70. Report the marginal value of each extra agent and show when a simpler graph is better.

Hard-mode extensions

71. Nested workflows, partial failures, unavailable providers, rate limits, conflicting agent outputs, stale memory, and workflow version migration.

72. An adversarial agent attempts to call a tool outside its declared scope or to smuggle instructions through a handoff; the orchestrator must block and surface it.

Safety boundary Every agent and tool must have explicit capabilities, budgets, and approval semantics. API keys belong in a secret broker; local-model execution must remain inside the declared workspace. The orchestrator must never turn a user goal into uncontrolled parallel side effects.

Expected deliverables

- 73. Workflow builder or API, provider adapters, agent registry, typed message/artifact store, and execution engine.
- 74. State/trace viewer with pause, resume, retry, cancel, and replay.
- 75. Workflow compiler, permission model, memory policy, and versioned configuration.
- 76. Benchmark/evaluation harness with single-agent and multi-agent comparisons.
- 77. Architecture, threat-model, workflow examples, and reproducibility report.

Why this stands out

This is the practical control surface for the agentic ecosystem: a system that makes multi-agent decomposition understandable and operational instead of hiding it inside prompts. Its strongest differentiator is the quality of the execution contract between agents.

TRACK 2

Cybersecurity

Detect, validate, and defend — with authorization built into the design.

This track is defensive and lab-bound. Participants are expected to build evidence-producing systems that protect people and systems without deanonymizing users or attacking third parties.

The track combines digital fraud and synthetic-media forensics, authorized security validation, and a unified defensive platform that treats phishing, availability attacks, and unknown network behavior as one operating picture.

CY-01

Digital Fraud & Synthetic-Media Forensics

Problem statement Build a privacy-preserving investigative framework that detects coordinated fraud, caller-ID spoofing, synthetic audio/video, manipulated documents, and account networks while producing confidence-scored, provenance-preserving reports rather than claims of identity or guilt.

AT A GLANCE

Design dimension	Participant expectation
Primary user	Fraud analysts, trust-and-safety teams, journalists, and authorized investigators.
Core inputs	Consented media, metadata, reports, public artifacts, and synthetic or de-identified case graphs.
Required outputs	Risk score, evidence graph, explainable findings, uncertainty, and review-ready report.
Hardest challenge	Combining multimodal detection and graph signals without overclaiming attribution.
Proof of quality	Held-out media/fraud cases, calibration, false-positive analysis, and provenance checks.

Core objective

Design an end-to-end system that helps an authorized analyst identify fraud patterns and manipulated media across a case bundle. The system may analyze voice, video, images, documents, messages, caller metadata, and relationships among accounts or events, but it must preserve privacy, show evidence, and distinguish detection from attribution.

Required capabilities and system blueprint

78. Multimodal media pipeline for voice spoofing, face/voice synchronization, image manipulation, document tampering, and synthetic-content indicators with model confidence and calibration.
79. Fraud-signal engine for velocity, reused contact points, coordinated account behavior, payment-pattern anomalies, device/session similarity, and report clustering using de-identified identifiers.
80. Evidence graph linking each finding to the original artifact, hash, timestamp, transformation, detector version, and human review state.
81. Privacy controls including redaction, pseudonymization, role-based access, retention limits, and a clear distinction between public evidence and restricted case data.
82. Analyst workflow for case intake, triage, alternative hypotheses, reviewer disagreement, escalation, and report export.
83. Calibration and threshold tools that make it possible to choose a conservative operating point for a high-cost false positive.
84. Adversarial robustness tests for compression, re-encoding, language/accent variation, background noise, and benign edits.

Minimum viable demonstration

85. Create a synthetic case bundle containing a scam call, a manipulated clip, a benign lookalike, and several coordinated artifacts.
86. Run the case through triage, show the evidence graph, and let a reviewer inspect why each signal contributed to the score.
87. Demonstrate that a missing or conflicting signal lowers confidence rather than being silently filled in by the model.
88. Export a report that a second analyst can reproduce from the same artifacts and detector versions.

Evaluation and benchmarks

89. Use relevant public media-forensics or anti-spoofing datasets such as NIST OpenMFC, ASVspoof, or organizer-provided synthetic cases; document licensing and preprocessing.
90. Measure detection precision/recall, calibration, subgroup performance, false-positive rate, evidence completeness, analyst time saved, and reviewer agreement.
91. Include a red-team set of benign content, edited-but-not-deceptive content, and prompt-injection text embedded in artifacts.

Hard-mode extensions

92. Cross-modal contradictions, low-quality media, multilingual evidence, unseen manipulation families, collusive account graphs, and missing metadata.

93. A case with strong synthetic-media evidence but no authorized identity data must remain a media-integrity finding, not an identity-resolution result.

Safety boundary Do not deanonymize VPN, Tor, or proxy users; infer a person's location; expose private data; or produce doxxing/attribution pipelines. Use synthetic, consented, or de-identified cases. Any law-enforcement workflow must be represented as a documented handoff to authorized processes, not automated identity discovery.

Expected deliverables

- 94. Multimodal ingestion and detector pipeline with versioned models.
- 95. Case graph, provenance store, analyst UI, privacy controls, and review workflow.
- 96. Calibration, bias, robustness, and held-out evaluation report.
- 97. Sample evidence-backed report and threat model.
- 98. Demo showing a blocked or downgraded attribution claim when evidence is insufficient.

Why this stands out

The value is not a flashy deepfake classifier. It is the defensible chain from artifact to signal to hypothesis to human decision, with privacy and uncertainty treated as core product requirements.

CY-02

Authorized Agentic Penetration-Testing Harness

Problem statement Build an agentic security-validation harness that tests only user-owned applications inside isolated labs, coordinates established security tools, verifies findings, proposes remediations, and produces an evidence-backed score without bypassing model safeguards or attacking live third-party systems.

AT A GLANCE

Design dimension	Participant expectation
Primary user	Developers and security teams validating applications they own or are authorized to test.
Core inputs	Source repository, container image, local service, test credentials, scope, and safety policy.
Required outputs	Findings, reproducible evidence, severity, remediation, retest result, and aggregate score.
Hardest challenge	High recall with low false positives while keeping tests bounded and non-destructive.
Proof of quality	Known-vulnerable lab targets, false-positive rate, patch retest, and audit trail.

Core objective

Create a sandbox-first harness that can inspect a submitted project, map its attack surface, run approved static and dynamic checks, verify findings, generate or recommend patches, and retest the result. LLM orchestration should add value in planning, interpretation, prioritization, and reporting rather than freehanding uncontrolled exploit behavior.

Required capabilities and system blueprint

99. Ingestion and scope declaration for a repository or containerized service, including explicit target ownership, allowed ports/routes, test accounts, and time/resource limits.
100. Isolated lab with network segmentation, disposable state, snapshots, read-only default, safe test data, secret isolation, and human approval for side-effectful checks.
101. Recon and static analysis including SAST, dependency/CVE scanning, secret scanning, API inventory, configuration checks, and test coverage mapping.
102. Tool orchestration around established scanners such as OWASP ZAP, Semgrep, Trivy/OSV-style dependency tools, and safe fuzzing in the closed target lab.
103. Agent planner that sequences test categories, tracks evidence, avoids duplicate work, and stops when scope, confidence, or risk thresholds are reached.
104. Finding verifier that distinguishes scanner output from confirmed evidence, records reproduction steps, assigns severity, and links the result to a code or configuration location.
105. Patch assistant and retest loop with human approval, regression checks, and a clear before/after security score.
106. Report generator that exposes false positives, unsupported assumptions, environmental constraints, and residual risk.

Minimum viable demonstration

107. Launch a known-vulnerable training application such as Juice Shop, WebGoat, or DVWA inside the lab and define a narrow target scope.
108. Run an agentic scan that combines static and dynamic evidence, then verify at least one finding with a reproducible safe test.
109. Generate a patch recommendation, apply it only after approval, rerun the relevant checks, and show whether the finding closes without regression.
110. Attempt a forbidden action outside the target scope and show the harness blocks it, logs the reason, and continues with safe checks.

Evaluation and benchmarks

111. OWASP Benchmark and known-vulnerable training targets; add organizer-provided hidden cases to prevent hard-coding.
112. Measure recall, precision, false-positive rate, severity calibration, time to verified finding, patch acceptance, retest success, human interventions, and policy violations.
113. Compare model-only interpretation with the full harness and report where deterministic tools were more reliable than generated payloads.

Hard-mode extensions

- 114. Authentication/session edge cases, API authorization, dependency conflicts, insecure configuration, SSRF-like lab scenarios, and multi-service applications.
- 115. Partially broken targets, noisy scan output, rate-limited services, flaky tests, and a patch that fixes one path but regresses another.

Safety boundary Testing must stay inside a closed lab or a user-owned target with explicit written scope. No live internet scanning, phishing, credential theft, persistence, destructive exploitation, social engineering, guardrail circumvention, or third-party reconnaissance. Default to detection/read-only mode.

Expected deliverables

- 116. Sandboxed target launcher, scope/policy engine, and audit log.
- 117. SAST/SCA/DAST orchestration adapters and evidence store.
- 118. Agent planning, finding verification, report generation, patch/retest workflow, and scorecard.
- 119. Evaluation report covering precision, recall, false positives, safety blocks, and reproducibility.
- 120. A lab demonstration with one verified finding and one blocked out-of-scope action.

Why this stands out

This challenge makes agentic security useful without turning the project into an offensive toolkit. The hard part is trustworthy verification and remediation under strict scope, not producing the most aggressive payload.

CY-03

Unified AI Cyber Defense Platform

Problem statement Build a SOC-style defensive platform that combines URL phishing detection, DDoS detection, and network anomaly detection into one explainable risk picture with alerts, historical analytics, and reversible response controls.

AT A GLANCE

Design dimension	Participant expectation
Primary user	Security analysts, platform operators, and small organizations without a full SOC.
Core inputs	URLs, DNS/SSL/redirect data, flow logs, packet statistics, login events, traffic patterns, and system metrics.
Required outputs	Risk scores, explanations, attack timeline, impacted entities, mitigation recommendation, and alert state.
Hardest challenge	Fusing heterogeneous signals into useful, low-noise, time-aware decisions.
Proof of quality	Replayed incidents, calibrated alerts, analyst workflow, and safe response simulation.

Core objective

Deliver a cohesive platform rather than three isolated notebooks. A web dashboard and backend should receive URL intelligence, traffic logs, and network behavior; a model service should analyze them; and a fusion layer should create a threat picture that a human analyst can understand, investigate, and act on.

Required capabilities and system blueprint

121. URL phishing module ingesting URL, domain age/entropy, SSL, DNS, redirect chain, lexical features, and reputation signals to return Safe, Suspicious, or Malicious with a 0–100 risk score and reasons.
122. DDoS module ingesting flow logs, packet statistics, source/ASN, protocol, volume, rate, and time windows to detect traffic surges, characterize severity, and recommend rate limiting, filtering, or upstream mitigation.
123. Network anomaly module learning normal behavior across devices/users/services and surfacing unknown deviations using Isolation Forest, autoencoders, robust statistics, or another justified method.
124. Spring Boot or equivalent backend with a FastAPI/model service, PostgreSQL for cases and history, Redis or a queue for streaming state, and a dashboard for alerts and investigation.
125. Fusion and explainability layer that correlates URL, traffic, identity, and system signals; exposes SHAP or feature contributions where appropriate; and preserves model/version provenance.
126. Response controls that are reversible and approval-gated: quarantine a test URL, apply a simulated block, create a ticket, notify a channel, or request analyst approval.
127. Role-based access for admin, analyst, and viewer; alert suppression/deduplication; trend analytics; and a replay mode for historical incidents.

Minimum viable demonstration

128. Ingest a suspicious URL and return an explainable score with evidence and redirect analysis.
129. Replay a labeled traffic window containing normal activity and a DDoS-like surge; show alerting, attacking sources, severity, and recommendation.
130. Replay an unknown behavior sequence that is not in the signature set and show anomaly score, affected device/user, and timeline.
131. Correlate the three modules into one incident view and demonstrate an approval-gated simulated response.

Evaluation and benchmarks

132. Use public or synthetic datasets such as phishing URL corpora, CIC-DDoS-style flow data, UNSW-NB15/TON-IoT-style anomaly data, and organizer replay traces with documented licenses.
133. Measure precision/recall, alert latency, false-positive rate, detection lead time, calibration, explanation usefulness, fusion lift, response safety, and operator time to triage.
134. Keep train/test separation by time or campaign to avoid leakage; include an unseen attack family and benign traffic spikes.

Hard-mode extensions

- 135. Distributed low-and-slow traffic, encrypted or missing features, novel domains, concept drift, noisy logs, clock skew, and correlated alerts from multiple sites.
- 136. A partial outage or false positive must not trigger an irreversible block automatically; the platform should degrade gracefully and preserve evidence.

Safety boundary Use synthetic or authorized traffic and assets only. Do not deploy blocking, quarantining, or packet manipulation against live networks without explicit approval. Never collect unnecessary personal data, claim certainty from a score, or provide a platform for surveillance or offensive operations.

Expected deliverables

- 137. Dashboard, backend/API, streaming or queued ingestion, model service, storage, and alerting.
- 138. Three modules with documented features, training/evaluation, and explanation outputs.
- 139. Fusion incident view, RBAC, replay mode, notification adapter, and reversible response simulation.
- 140. Dataset lineage, model card, threat model, and evaluation report.
- 141. End-to-end demo with one phishing, one DDoS, and one unknown-anomaly scenario.

Why this stands out

The standout contribution is the operating picture: one platform that protects users from malicious links, safeguards availability, and identifies unknown behavior while remaining explainable and safe enough for a small organization to operate.

TRACK 3

SDG

Turn intelligence into accountable public-good operations.

These challenges use agentic systems to make civic response, disaster management, and mission operations more accessible, resilient, and measurable. The human outcome matters as much as the model.

The SDG track follows the three reference ideas supplied for the handbook: Community Redressal Planner, Disaster Management System with satellite mapping and natural-calamity detection, and Space Mission Operations Automator.

SDG-01

Community Redressal Planner

Problem statement Build a multilingual, privacy-aware civic redressal system that turns resident complaints into deduplicated, prioritized, accountable workflows with transparent status, escalation, and measurable service-level outcomes.

AT A GLANCE

Design dimension	Participant expectation
Primary user	Residents, community organizations, and public-service teams.
Core inputs	Text/voice complaints, location, category, evidence, service directory, and SLA rules.
Required outputs	Triage, route, priority, status, explanation, escalation path, and resolution analytics.
Hardest challenge	Making routing useful without making automated eligibility or denial decisions.
Proof of quality	Routing accuracy, multilingual performance, duplicate detection, accessibility, and fairness review.

Core objective

Design a community-facing planner that helps people report issues — such as broken infrastructure, sanitation, safety hazards, access barriers, or local service gaps — and helps authorized teams route, prioritize, coordinate, and close them. The system should surface patterns and bottlenecks while preserving the resident's agency and privacy.

Required capabilities and system blueprint

142. Accessible intake across web, low-bandwidth mobile, SMS/voice simulation, multilingual text, and optional image/audio evidence.
143. Classifier and entity extraction for issue category, location, urgency, affected service, and duplicate/similar complaints with confidence shown to reviewers.
144. Routing and planning engine using department/service maps, jurisdiction rules, workload, SLA, geography, and escalation paths.
145. Resident view with reference number, plain-language status, expected next step, appeal/correction path, and consent/retention controls.
146. Operations dashboard with map, queue, SLA risk, bottlenecks, recurring hotspots, and cross-department coordination.
147. Fairness and accessibility checks across language, neighborhood, disability-access needs, and reporting channel; allow human override with reason.
148. Evidence/provenance and privacy layer that redacts unnecessary personal data and prevents a complaint from becoming a public doxxing surface.

Minimum viable demonstration

149. Submit complaints in two languages with incomplete information and show clarification prompts that do not block urgent escalation.
150. Cluster duplicate reports, route them to the correct service, and show why a human reviewer can change the route or priority.
151. Simulate an SLA breach and demonstrate escalation, resident notification, and an audit trail.
152. Produce neighborhood-level trend analytics without exposing individual reporters.

Evaluation and benchmarks

153. Evaluate category and route accuracy, duplicate precision/recall, multilingual parity, accessibility task success, SLA prediction, complaint-resolution time, and human override rate.
154. Use synthetic or properly licensed civic issue data with a documented privacy model; test on edge cases such as slang, code-switching, ambiguous locations, and duplicate images.
155. Measure whether the planner improves accountable handoff rather than merely producing better summaries.

Hard-mode extensions

156. Noisy voice transcription, conflicting reports, changing jurisdiction, recurring emergencies, low connectivity, missing addresses, and malicious or abusive submissions.

157. Show a safe fallback when the system cannot determine priority or destination; uncertainty must prompt human review, not silent rejection.

Safety boundary Do not use the system for political profiling, surveillance, automated benefits denial, retaliation, or public exposure of complainants. Protect identity and location data, support correction and appeal, and keep final eligibility, enforcement, and emergency decisions with authorized humans.

Expected deliverables

- 158. Resident intake and status experience with accessibility considerations.
- 159. Triage, deduplication, routing, SLA, escalation, and analytics services.
- 160. Privacy, consent, redaction, fairness, and audit controls.
- 161. Synthetic evaluation set, model card, service blueprint, and impact report.
- 162. End-to-end demonstration from report to accountable resolution.

Why this stands out

This problem is difficult because the outcome is organizational, not merely informational. The system must connect people to action, preserve trust, and show whether it improved response quality for communities that are often least well served by existing channels.

SDG-02

Disaster Management System with Satellite Mapping and Natural-Calamity Detection

Problem statement Build a resilient disaster-management system that combines satellite imagery, weather and sensor feeds, geospatial mapping, and agentic planning to detect hazards, estimate impact, prioritize response, and support evacuation and resource decisions in simulation.

AT A GLANCE

Design dimension	Participant expectation
Primary user	Emergency operations teams, local authorities, NGOs, and community responders.
Core inputs	Satellite imagery, weather, river/soil/sensor data, maps, population layers, and resource inventory.
Required outputs	Hazard map, severity, affected zones, confidence, response plan, and update timeline.
Hardest challenge	Making time-sensitive decisions under missing, delayed, and conflicting observations.
Proof of quality	Held-out events, lead time, spatial accuracy, calibration, and plan feasibility.

Core objective

Develop a simulation-first command system for floods, wildfires, cyclones, landslides, earthquakes, or other natural hazards. It should ingest changing observations, detect or update the hazard footprint, combine it with exposure and critical infrastructure, propose response actions, and show uncertainty and human approval points.

Required capabilities and system blueprint

163. Geospatial data layer for satellite tiles, elevation, roads, shelters, hospitals, population, weather, river gauges, and sensor events with timestamp/version tracking.
164. Natural-calamity detection and change-detection models appropriate to the chosen hazard; support cloud cover, missing tiles, geolocation error, and noisy sensor data.
165. Impact estimation for affected population, roads, utilities, agriculture, facilities, and potential cascading failures.
166. Map and timeline interface showing hazard polygons, confidence, new/changed observations, data gaps, and analyst annotations.
167. Agentic response planner that proposes evacuation routes, shelter allocation, medical/logistics priorities, volunteer tasks, and update cadence within declared constraints.
168. Offline/low-bandwidth mode, cache/queue behavior, alert prioritization, and human confirmation before public warnings or resource commitments.
169. Post-event evaluation and learning loop that compares predictions to verified outcomes without silently changing operational policy.

Minimum viable demonstration

170. Replay a historical or synthetic event with sequential satellite and sensor updates; show the map evolving over time.
171. Introduce a missing data window and show how the system communicates uncertainty and chooses a safe fallback.
172. Generate a response plan for a defined population and resource inventory, then let a reviewer approve, edit, or reject each action.
173. Produce a post-event report comparing detection lead time, affected-area estimate, and resource-plan feasibility against ground truth.

Evaluation and benchmarks

174. Use properly licensed or public datasets such as flood/fire imagery, disaster mapping benchmarks, weather/sensor replays, and organizer simulations.
175. Measure spatial IoU or detection accuracy, lead time, calibration, false-alarm rate, missing-data robustness, route feasibility, plan completion, and decision-maker usability.
176. Keep geographic and temporal splits to avoid memorizing the same event; include an unseen hazard or region where practical.

Hard-mode extensions

- 177. Night/cloud-obscured imagery, rapidly changing hazards, sensor disagreement, road closures, communication outages, cascading failures, and simultaneous events.
- 178. A public alert, evacuation command, or resource dispatch must remain approval-gated; the demo must make it obvious when a plan is a recommendation rather than a confirmed instruction.

Safety boundary Use simulation, historical data, or authorized feeds only. Do not issue real emergency warnings, publish precise vulnerable-location data, or direct people into unsafe areas. Preserve uncertainty and require human authority for life-safety decisions.

Expected deliverables

- 179. Geospatial ingestion, feature store, change-detection/anomaly models, and map/timeline dashboard.
- 180. Response planner with constraints, resource inventory, routes, approvals, and offline behavior.
- 181. Historical/synthetic event replay, evaluation report, and data-provenance notes.
- 182. Threat model covering false alarms, missed events, outages, and sensitive location data.
- 183. Simulation demo with a missing-data or conflicting-sensor scenario.

Why this stands out

The strongest entry will show not just a colored hazard map but a resilient decision loop: observe, estimate, explain uncertainty, plan within constraints, ask for approval, and learn from the outcome.

SDG-03

Space Mission Operations Automator

Problem statement Build a simulation-first mission-operations system that plans activities, monitors telemetry, detects anomalies, schedules resources, and proposes procedure steps while keeping every real or irreversible command behind explicit authority and verification.

AT A GLANCE

Design dimension	Participant expectation
Primary user	Mission planners, flight controllers, and spacecraft-operations teams in a simulator.
Core inputs	Mission goals, timeline, telemetry, constraints, procedures, resources, and simulated faults.
Required outputs	Plan, constraint checks, anomaly diagnosis, procedure recommendation, and operator trace.
Hardest challenge	Coordinating time, resources, safety constraints, and partial observability without unsafe automation.
Proof of quality	Digital-twin replay, fault injection, constraint satisfaction, and operator approval workflow.

Core objective

Create an operations automator for a simulated satellite, rover, observatory, or interplanetary mission. The system should turn mission goals into a schedule, monitor telemetry, detect and characterize anomalies, propose safe procedure steps, manage communication windows and resources, and preserve the operator as the final authority for commands.

Required capabilities and system blueprint

184. Mission model or digital twin for spacecraft state, instruments, power, thermal, data storage, pointing, communications, and time-dependent constraints.
185. Planner that schedules observations, downlinks, maintenance, calibration, and safe-mode transitions while respecting resource and precedence constraints.
186. Telemetry pipeline with baseline modeling, limit checks, trend analysis, anomaly detection, alarm correlation, and uncertainty-aware diagnosis.
187. Procedure engine that retrieves versioned runbooks, checks preconditions, simulates effects, and proposes but does not silently execute commands.
188. Fault-injection harness for sensor drift, packet loss, power/thermal excursions, communication delay, conflicting telemetry, and actuator failure.
189. Operator console with timeline, state, evidence, approvals, command preview, rollback/abort semantics, and post-event review.
190. Mission memory for configuration, prior anomalies, procedures, and decisions with strict versioning and authority boundaries.

Minimum viable demonstration

191. Load a mission plan, schedule a constrained observation/downlink sequence, and show why the planner selected the order.
192. Inject a telemetry anomaly and show detection, competing hypotheses, evidence, and a recommended safe procedure.
193. Require operator approval for a command and show that the simulator previews and validates it before execution.
194. Replay the event and compare automated recommendation, operator decision, and simulated mission outcome.

Evaluation and benchmarks

195. Use an open mission simulator, synthetic telemetry, or organizer-provided digital-twin tasks; document assumptions and units.
196. Measure schedule feasibility, resource violations, anomaly precision/recall, time to detection, diagnosis ranking, safe-procedure success, and operator workload.
197. Include fault-injection tests, unseen combinations of faults, delayed observations, and recovery from a failed plan.

Hard-mode extensions

- 198. Conflicting sensors, missing telemetry, long communication delays, multiple simultaneous faults, changing mission priorities, and limited onboard resources.
- 199. No system should transmit real commands or interface with a live spacecraft. The challenge is deliberately simulation-first and approval-gated.

Safety boundary Keep the entire system in a simulator or approved lab. Never connect an autonomous agent to live spacecraft, flight hardware, or mission credentials. Command generation must be separated from command authorization, and every simulated action must be replayable.

Expected deliverables

- 200. Mission digital twin or simulator with telemetry generator and fault injection.
- 201. Planner, anomaly detector, procedure/retrieval engine, operator console, and trace store.
- 202. Constraint, safety, authority, and versioning model.
- 203. Replayable evaluation suite and fault-analysis report.
- 204. Demo showing an anomaly, approval-gated procedure, and recovery outcome.

Why this stands out

Space operations are a compelling test of agentic reliability because the system must reason over time, constraints, partial observability, and consequences. A simulator makes the challenge ambitious while keeping experimentation safe and repeatable.

TRACK 4

Web3 Agentic Payment Applications

Make software resources purchasable at the moment an agent needs them.

These application challenges use the standard x402 client/server payment flow to build useful pay-per-use products. The system should feel like a service first and a blockchain demo second.

Choose one application and demonstrate an agent paying for a resource at request time. Use a testnet, simulated facilitator, or capped sandbox. Document the exact scheme, network, wallet policy, and failure behavior.

PAY-01

Pay-per-Use AI & Data API Marketplace

Problem statement Build a marketplace where agents discover, evaluate, and pay for AI/data APIs on a per-request or usage-capped basis through x402, then receive a verifiable result, receipt, and cost-aware usage record.

AT A GLANCE

Design dimension	Participant expectation
Primary user	Agents and developers consuming specialized data, inference, or transformation APIs.
Core inputs	Agent request, resource catalog, payment requirements, wallet policy, and provider response.
Required outputs	Paid result, receipt, provenance, cost, latency, and refund/error semantics.
Hardest challenge	Making autonomous purchase decisions trustworthy under price, quality, and failure uncertainty.
Proof of quality	Successful exact/up to flows, provider comparison, budget enforcement, and replayable receipts.

Core objective

Create a service marketplace in which agents can find resources such as OCR, translation, embeddings, geocoding, risk scoring, domain data, or specialized inference, compare price/quality/latency, pay at the HTTP boundary, and receive a result that can be used in a larger workflow. The product should make the payment lifecycle observable and keep agents from spending beyond policy.

Required capabilities and system blueprint

205. Resource catalog with provider identity, capabilities, input/output schema, price, network, scheme, rate limits, freshness, quality metrics, and terms.
206. x402 client that handles HTTP 402, selects a compatible payment requirement, signs a capped payment payload, retries safely, and attaches the payment response to the trace.
207. Provider adapter and facilitator integration or simulator for verification and settlement; support fixed-price exact and, where justified, usage-capped up to flows.
208. Agent purchasing policy with per-request, per-task, per-provider, and daily budgets; allowlists; quality thresholds; fallback providers; and approval escalation.
209. Result provenance linking paid resource, provider version, input hash, output hash, timestamp, cost, and settlement/receipt state.
210. Marketplace scoring that measures quality and total cost rather than ranking only on price; support provider failure and a clear retry/refund policy.
211. Developer UX: SDK/CLI, catalog browser, test mode, sample provider, and a trace viewer that shows the 402 exchange without exposing private keys.

Minimum viable demonstration

212. Register two providers with different cost/latency/quality profiles and let an agent select between them within a budget.
213. Run a fixed-price request and a usage-capped request in a sandbox; show requirements, signed payload, verification, settlement, response, and receipt.
214. Force a provider failure or invalid payment and demonstrate bounded retry/fallback without double spending.
215. Export a per-task cost and provenance report that a judge can replay.

Evaluation and benchmarks

216. Use the x402 introduction, client/server, facilitator, and buyer/seller quickstart flow as the protocol baseline; cite the chosen implementation version in the README.
217. Measure task success, payment success, duplicate-payment prevention, budget violations, provider-selection quality, result quality, cost, latency, and receipt completeness.
218. Use a held-out provider or schema to test whether the client is protocol-driven rather than hard-coded to one endpoint.

Hard-mode extensions

- 219. Provider disappears after 402, price changes, stale catalog entries, malformed payment requirements, unsupported network/scheme, rate limits, and partial settlement.
- 220. A malicious provider attempts to inject instructions into the resource description or result; the agent must treat content as data and keep payment policy separate.

Safety boundary Use sandbox/testnet wallets, capped budgets, and simulated or approved facilitators. Never expose a seed phrase or unrestricted signing key, automatically purchase unknown services, or make a real-world financial decision solely from untrusted provider content.

Expected deliverables

- 221. Catalog, provider SDK, x402 client, facilitator/testnet adapter, and policy engine.
- 222. Sample AI/data providers with exact and/or upto payment flows.
- 223. Agent trace, receipt/provenance store, budget dashboard, and failure simulation.
- 224. Protocol compliance, security, cost, and quality evaluation report.
- 225. End-to-end demo with provider fallback and a blocked overspend.

Why this stands out

This is the cleanest application of agentic commerce: an agent can buy a useful capability exactly when needed, without account creation or a pre-negotiated billing relationship. The hard part is the decision and evidence layer around the payment.

PAY-02

Agentic Research and Content Paywall

Problem statement Build an evidence-aware content or research service that lets agents pay per document, query, citation bundle, or structured extraction through x402 while preserving provenance, access rights, and user budget controls.

AT A GLANCE

Design dimension	Participant expectation
Primary user	Research agents, analysts, publishers, and specialized knowledge providers.
Core inputs	Query, content metadata, access tier, payment requirement, user policy, and citation need.
Required outputs	Authorized content/extract, citations, receipt, usage rights, and confidence notes.
Hardest challenge	Aligning micropayment, licensing, citation quality, and agent decisions without leaking private content.
Proof of quality	Correct access, citation completeness, rights enforcement, and spend-aware research outcomes.

Core objective

Create a pay-per-use knowledge product: a publisher, archive, data room, market-intelligence feed, premium report service, or document-extraction API. An agent should be able to preview metadata, decide whether the resource is worth the cost, pay via x402, retrieve the permitted content, and preserve the citation and usage terms in its final answer.

Required capabilities and system blueprint

226. Content/resource catalog with title, abstract, freshness, license tier, source provenance, price, response shape, and permitted downstream use.
227. Preview-before-pay flow so an agent can compare expected value, duplicate coverage, and source quality without receiving the paid content for free.
228. x402 resource server for per-document, per-query, or per-extraction charges; support exact for fixed items and upto for bounded usage where justified.
229. Agent research policy for source quality, maximum spend, duplicate avoidance, citation requirements, geographic/language constraints, and human approval for high-cost access.
230. Citation/provenance layer that links each claim to a paid artifact, page/section or structured field, access timestamp, and license restrictions.
231. Entitlement and replay controls that prevent a paid result from being silently reused beyond its declared scope; make caching and retention explicit.
232. Publisher analytics showing conversion, failed payments, content quality feedback, and suspicious access patterns without exposing reader identity unnecessarily.

Minimum viable demonstration

233. Let an agent compare three content items, purchase one, and produce a cited answer with a cost/benefit trace.
234. Run an extraction request with an upto budget and show settlement for actual usage plus a receipt.
235. Attempt to cite or reuse content outside the permitted scope and demonstrate a policy warning or blocked output.
236. Introduce a stale or contradictory source and show that the agent reports the conflict rather than buying more blindly.

Evaluation and benchmarks

237. Measure citation precision/completeness, access/payment success, duplicate-purchase avoidance, source-quality lift, budget adherence, unsupported-claim rate, and latency.
238. Evaluate with public-domain, synthetic, or properly licensed content; include expired, corrected, paywalled, and malformed documents.
239. Test protocol compatibility with a second client and a provider whose metadata schema is different from the reference implementation.

Hard-mode extensions

- 240. Content update after preview, partial extraction, provider outage, conflicting rights metadata, repeated queries, and agent prompt injection embedded in documents.
- 241. A user must be able to inspect what was purchased, why it was purchased, how much it cost, and what downstream use is allowed.

Safety boundary Use public-domain, synthetic, or licensed content and a testnet/simulated payment rail. Do not scrape paywalled material without authorization, bypass access controls, redistribute copyrighted content, or let a document's instructions control the payment or research policy.

Expected deliverables

- 242. Content catalog, preview service, paywall/resource server, x402 payment adapter, and entitlement layer.
- 243. Research-agent integration with budget, citation, and rights policies.
- 244. Receipt/provenance view and publisher analytics.
- 245. Evaluation report for citation quality, cost, rights enforcement, and adversarial content.
- 246. Demo with an exact purchase, bounded usage purchase, conflict, and blocked reuse.

Why this stands out

It turns a research workflow into a transparent market: knowledge providers can charge for a precisely defined unit, and agents can decide whether the evidence is worth buying while carrying provenance into the final answer.

PAY-03

Machine-to-Machine Compute and Tool Exchange

Problem statement Build an agent-to-service exchange in which software agents discover and pay for short-lived compute, rendering, simulation, validation, or transformation jobs through x402, with quotas, result integrity, and automatic cleanup.

AT A GLANCE

Design dimension	Participant expectation
Primary user	Agents that need bursty compute or specialist tools without provisioning a long-lived account.
Core inputs	Job specification, resource offer, payment requirement, constraints, data artifact, and result policy.
Required outputs	Completed artifact, job log, receipt, cost, resource usage, and cleanup confirmation.
Hardest challenge	Binding payment to a bounded job and proving what was executed and delivered.
Proof of quality	Correct metering, result integrity, failure recovery, and no resource leakage.

Core objective

Design a marketplace or direct exchange for agent-consumed tools such as image/video rendering, model inference, simulation, test execution, document conversion, geospatial transformation, or data validation. The agent should choose a provider, pay for a bounded job, monitor progress, verify the result, and clean up temporary data.

Required capabilities and system blueprint

247. Job schema with input/output contracts, resource class, max duration, data retention, price model, supported network/scheme, and cancellation behavior.
248. x402 resource server that can price a fixed job or cap usage; keep payment and job execution state separate and reconciled through a receipt.
249. Ephemeral worker pool using containers or microVMs, resource quotas, input scanning, restricted network, temporary credentials, and automatic cleanup.
250. Agent planner that estimates value and cost, selects a provider, supplies only the minimum data, handles progress/failure, and verifies output integrity.
251. Metering and result attestation showing requested versus actual CPU/GPU/time/bytes, tool version, input/output hashes, and settlement state.
252. Provider reputation and scheduling layer based on quality, latency, price, and prior failure rate rather than an opaque single score.
253. Cancellation, retry, refund, and dispute simulation for jobs that fail before completion or return an invalid artifact.

Minimum viable demonstration

254. Submit a bounded job to two provider simulators and choose based on price, latency, and required quality.
255. Run an exact-price job and an upto/metered job; compare quoted versus actual cost and verify the receipt.
256. Inject a worker timeout and an incorrect output hash; show safe retry/compensation and cleanup.
257. Inspect the worker trace without exposing the agent's secret or unrelated tenant data.

Evaluation and benchmarks

258. Measure job completion, result-integrity rate, metering accuracy, cleanup success, cost/latency, retry behavior, provider-selection quality, and data exposure incidents.
259. Use synthetic artifacts and disposable workers; document the sandbox boundary and do not rely on a public cloud account with unrestricted credentials.
260. Test schema evolution, provider differences, duplicate requests, and an unseen job type through a compatibility adapter.

Hard-mode extensions

261. Large inputs, partial results, worker crash, network loss, cost overrun, provider misreporting, malicious artifact, and repeated job submission.
262. The worker must never be allowed to access host files, unrelated tenant data, long-lived secrets, or an unrestricted network. A job must be cancelable and auditable.

Safety boundary Use local containers, testnets, simulated payments, or capped sandbox accounts. Do not rent uncontrolled compute, process personal data without consent, or accept arbitrary code execution outside a disposable worker with explicit limits.

Expected deliverables

- 263. Job catalog and provider adapters with x402 resource server.
- 264. Ephemeral worker/sandbox, metering, input/output integrity, and cleanup service.
- 265. Agent-side scheduler, budget policy, retry/compensation flow, and receipt dashboard.
- 266. Failure, security, metering, and result-quality evaluation.
- 267. Demo with two providers, one timeout, and one invalid-result case.

Why this stands out

The challenge makes agentic payments operational: payment is attached to a bounded, measurable unit of work, and the system must prove both that money was authorized and that the requested artifact was actually delivered.

TRACK 5

Web3 Agentic Payment Infrastructure

Build the rails that let many paid services compose safely.

These infrastructure challenges sit above one x402 resource. They coordinate providers, wallets, facilitators, networks, budgets, receipts, and workflow state — while deliberately avoiding custody of real funds in the participant demo.

Choose one composite/orchestrator challenge. Judges will look closely at policy, replay protection, settlement correctness, failure recovery, and the quality of the trace across multiple services.

INF-01

Multi-Provider Agent Payment Router & Treasury

Problem statement Build a policy-driven router that decomposes an agent task across multiple x402 providers, selects routes by price/quality/latency, enforces budgets, handles fallbacks, and reconciles every payment and result without exposing an unrestricted wallet.

AT A GLANCE

Design dimension	Participant expectation
Primary user	Agent platforms and teams consuming several paid capabilities in one task.
Core inputs	Goal, provider catalog, budgets, wallet policy, quality constraints, and workflow graph.
Required outputs	Route plan, paid sub-results, receipts, reconciliation, and final cost/quality report.
Hardest challenge	Coordinating multiple providers and failures while preserving spend and result integrity.
Proof of quality	Budget compliance, route quality, no duplicate settlement, and replayable reconciliation.

Core objective

Create the payment control plane for a multi-step agent. For example, a research request may need search, extraction, translation, ranking, and verification from different x402 resources. The router should plan the calls, choose providers, authorize bounded payments, merge results, retry safely, and reconcile quotes, actual charges, receipts, and final outcome.

Required capabilities and system blueprint

- 268. Provider discovery and normalized capability/price/quality schema across networks, schemes, facilitators, currencies/tokens, and response contracts.
- 269. Task planner that maps a high-level goal to paid sub-requests, supports parallel fan-out and dependency-aware joins, and avoids purchasing redundant evidence.
- 270. Treasury/policy layer with per-task, provider, network, user, and time-window budgets; allowlists; risk thresholds; approvals; and spend reservation.
- 271. Route optimizer balancing price, latency, quality, reliability, data locality, and settlement risk; expose why a route was selected.
- 272. Safe wallet interface using scoped signing, transaction simulation, nonce/idempotency controls, and no access to a raw seed phrase in the agent context.
- 273. Reconciliation ledger joining request, 402 requirements, payment payload, verification, settlement, response, receipt, refunds/compensation, and result hashes.
- 274. Fallback and recovery semantics for provider outage, payment rejection, partial workflow completion, stale quotes, and duplicate retries.

Minimum viable demonstration

- 275. Run a five-step task across at least three providers with one parallel branch and one fallback.
- 276. Force one provider or facilitator to fail after a 402 and show that the router does not double spend or lose task state.
- 277. Exceed a sub-budget and demonstrate pause/approval rather than silent overspend.
- 278. Export an end-to-end reconciliation view that lets a judge tie every paid request to a result or a declared failure.

Evaluation and benchmarks

- 279. Measure end-to-end task success, route optimality, budget adherence, duplicate-payment rate, settlement correctness, fallback recovery, latency, and cost.
- 280. Test exact and upto schemes, multiple provider schemas, a simulated multi-network catalog, and stale or conflicting price metadata.
- 281. Evaluate with held-out provider combinations so the router cannot rely on a fixed order or one preferred service.

Hard-mode extensions

- 282. Parallel workflow failure, price change between quote and payment, provider result poisoning, network mismatch, rate limits, partial settlement, and ledger replay.

283. A provider or result must never be allowed to rewrite the payment policy, raise a budget, or gain access to unrelated wallet scopes.

Safety boundary Use testnet/simulated funds, scoped wallet APIs, and hard caps. Do not implement a general-purpose custodial wallet, hide exchange-rate risk, or automatically route real money based on untrusted provider claims.

Expected deliverables

- 284. Provider catalog, normalized route model, task graph, and payment policy engine.
- 285. Scoped wallet/signing adapter, x402 client, facilitator/testnet integration, and idempotency controls.
- 286. Reconciliation ledger, receipt dashboard, fallback/recovery engine, and trace export.
- 287. Route-quality, budget, settlement, security, and failure evaluation.
- 288. Composite demo with three providers, one outage, and one approval-gated overspend.

Why this stands out

This problem turns isolated pay-per-use calls into a real agentic economy control plane. Its difficulty lies in orchestration correctness: the system must be cost-aware, failure-aware, and auditable across many individual payment events.

INF-02

Facilitator and Settlement Mesh

Problem statement Build a non-custodial verification and settlement mesh that can accept x402 payment payloads across supported schemes or networks, route them to appropriate facilitators, prevent replay and duplicate settlement, and return consistent receipts and errors.

AT A GLANCE

Design dimension	Participant expectation
Primary user	Resource servers and agent platforms that need reliable payment verification/settlement.
Core inputs	Payment requirements, signed payloads, scheme/network, facilitator health, and settlement result.
Required outputs	Verification decision, settlement outcome, receipt, error semantics, and audit record.
Hardest challenge	Correctness under retries, duplicates, network differences, partial failure, and provider drift.
Proof of quality	Protocol tests, replay resistance, consistent failure handling, and observable settlement state.

Core objective

Implement the infrastructure layer that resource servers can call to verify and settle x402 payments without each one maintaining its own blockchain integration. The mesh should route by scheme and network, monitor facilitator health, enforce idempotency and replay protection, normalize receipts, and make failure states explicit. Keep the implementation non-custodial and simulator/testnet-first.

Required capabilities and system blueprint

289. Compatibility layer for payment requirements, payload validation, schemes, network identifiers, and versioned protocol behavior.
290. Facilitator registry with capability discovery, health checks, latency/error metrics, routing rules, and circuit breakers.
291. Verification service that validates payload shape, amount, recipient, resource, expiry, nonce/idempotency, network, and declared constraints before fulfillment.
292. Settlement service with idempotency keys, replay/duplicate detection, confirmation tracking, timeout handling, and explicit pending/failed/succeeded states.
293. Receipt service returning normalized payment/settlement evidence, facilitator/network references, resource identifier, and timestamp without exposing sensitive keys.
294. Observability and dispute/reconciliation console that can trace a payment from 402 response through verification, settlement, resource fulfillment, and final response.
295. Extension/plugin interface for receipts, discovery, gas sponsorship, or authentication that does not silently change core payment semantics.

Minimum viable demonstration

296. Run exact-payment verification and settlement through two facilitator simulators with different latency and failure profiles.
297. Submit the same payload twice concurrently and show that the resource is fulfilled only once in the simulator.
298. Simulate a network confirmation delay, facilitator outage, and malformed payload; show consistent responses and trace state.
299. Generate a receipt that a second service can validate without trusting the dashboard UI.

Evaluation and benchmarks

300. Use the official x402 facilitator flow and protocol tests as the baseline; add organizer tests for replay, duplicate settlement, malformed requirements, and cross-network routing.
301. Measure verification correctness, duplicate/replay rejection, settlement finality handling, receipt completeness, failover time, false rejection rate, and operator visibility.
302. Include a conformance matrix for every scheme/network/facilitator combination actually supported; do not imply support beyond tested combinations.

Hard-mode extensions

- 303. Concurrent duplicate settlement, delayed confirmations, facilitator disagreement, chain reorg simulation, network mislabeling, expired payload, and resource-server retry storms.
- 304. A facilitator must never become a hidden custodian or a single point of silent trust; document trust assumptions and fail closed when verification is uncertain.

Safety boundary Use local simulators, testnets, or capped sandbox wallets. Do not custody user funds, store seed phrases, or present a simulated receipt as real settlement. The mesh must fail closed on ambiguity and must not provide a tool for evading sanctions, fraud checks, or payment-provider controls.

Expected deliverables

- 305. Protocol parser/validator, facilitator registry, health/routing layer, and verification API.
- 306. Settlement state machine with idempotency, replay protection, confirmation tracking, and receipt service.
- 307. Simulator/testnet adapters, conformance tests, dashboards, and audit exports.
- 308. Threat model covering duplicate settlement, payload manipulation, facilitator failure, and data privacy.
- 309. Failure-injection demo and support matrix.

Why this stands out

Payment infrastructure is where small correctness gaps become systemic failures. This challenge rewards the engineering discipline behind a trustworthy x402 ecosystem: precise validation, non-custodial settlement, explicit state, and excellent recovery behavior.

INF-03

Composite Agentic Commerce Orchestrator

Problem statement Build a higher-level orchestrator that lets an agent negotiate and execute a multi-step service workflow — discovery, quotes, x402 payments, conditional fulfillment, verification, and final settlement — while keeping business rules, budgets, and approvals visible.

AT A GLANCE

Design dimension	Participant expectation
Primary user	Agents completing a goal that requires several independent paid services.
Core inputs	Goal, workflow constraints, service offers, user policy, payment options, and approval rules.
Required outputs	Workflow plan, conditional sub-results, payments, final artifact, and complete audit trail.
Hardest challenge	Making a multi-party agent workflow trustworthy when each step can change what is needed next.
Proof of quality	Plan correctness, conditional state, spend control, fulfillment verification, and replay.

Core objective

Build an orchestrator for a composite task such as ‘research a market, translate the evidence, validate the numbers, and generate a report’ or ‘design an asset, run simulation, validate constraints, and deliver a package.’ Each sub-service may be paid through x402, but the orchestrator must coordinate the workflow, approvals, conditions, and receipts without pretending that the base protocol itself provides escrow or business-level guarantees.

Required capabilities and system blueprint

310. Service discovery and offer normalization for capabilities, schemas, price/scheme/network, quality, terms, delivery conditions, and expiration.
311. Workflow compiler that turns a goal into a directed graph with prerequisites, parallel branches, conditional edges, human approvals, and compensation/stop semantics.
312. Quote and budget manager that reserves, releases, or escalates spend; distinguishes estimated from settled cost; and prevents a sub-result from silently raising the overall budget.
313. x402 payment adapter for each paid edge with exact/upto support as appropriate, scoped wallet calls, idempotency, and receipt capture.
314. Fulfillment verifier that checks schema, quality, provenance, hashes, policy, and dependency conditions before allowing the next payment or final delivery.
315. Human-in-the-loop checkpoints for high-cost, high-risk, external-publication, or ambiguous steps with pause/resume/review semantics.
316. Workflow ledger and trace across agent decisions, offers, approvals, 402 exchanges, payments, results, retries, and final artifact version.
317. Simulation/test harness for partial completion, refunds/compensation, provider disputes, and user cancellation.

Minimum viable demonstration

318. Build a three-provider workflow with one branch that can be skipped when a quality condition is already met.
319. Show a quote, approval, x402 payment, result verification, and conditional next step in one trace.
320. Cancel the workflow after one paid step and show what is delivered, what is not purchased, and how the ledger closes.
321. Replay the workflow with a changed provider offer and show which decisions change and which policy remains fixed.

Evaluation and benchmarks

322. Evaluate workflow success, conditional-edge correctness, budget adherence, result-verification accuracy, duplicate-payment rate, human approval quality, and final artifact quality.
323. Use a provider simulator with exact and upto offers, stale quotes, partial results, and conflicting quality metadata.
324. Measure whether orchestration improves task outcomes over a single-agent baseline and whether the trace makes the workflow understandable to a reviewer.

Hard-mode extensions

- 325. Provider changes offer after approval, dependency failure, conflicting results, cancellation, partial refund simulation, stale artifact, and malicious instructions in a provider response.
- 326. The orchestrator must not claim escrow, guaranteed delivery, or legal settlement unless it actually implements and documents a separate mechanism; in the participant demo, use explicit simulation semantics.

Safety boundary Use testnet/simulated funds, explicit budget caps, and human approval for high-impact steps. Do not create an uncontrolled autonomous purchasing agent, use real public funds, make legal/financial commitments, or treat untrusted offer text as policy.

Expected deliverables

- 327. Offer/catalog layer, workflow compiler, state machine, budget/policy engine, and approval UI.
- 328. x402 client/facilitator adapters with scoped wallets, receipts, and idempotency.
- 329. Fulfillment verification, workflow ledger, trace viewer, and simulation harness.
- 330. Comparison against single-agent and non-orchestrated baselines.
- 331. Composite demo with conditional branch, cancellation, and replay.

Why this stands out

This is the capstone infrastructure problem: it asks whether agentic commerce can become a dependable workflow rather than a sequence of opaque purchases. The differentiator is the contract between payment, fulfillment, policy, and evidence.

Submission rules

The rules are designed to make submissions comparable and safe to evaluate. Teams may use commercial APIs, local models, open-source tools, and managed services, but every dependency must be disclosed and reproducible within reasonable limits.

Required submission package

332. Source repository or an auditable code bundle with a clear license and ownership statement.
333. README with setup, environment variables, model/provider configuration, seed data, and the shortest path to a working demo.
334. Architecture diagram, threat model, data-flow description, and list of external services.
335. Evaluation report with baseline comparison, task definitions, metrics, cost/latency, and known limitations.
336. Demo script and a short recorded walkthrough or live-demo plan. Keep one prepared scenario and one held-out scenario.
337. Sample outputs and trace exports that do not contain secrets, personal data, or unlicensed source material.
338. A statement of what is simulated, what is real, what requires approval, and what is intentionally out of scope.

Reproducibility checklist

Check	Include
Environment	Runtime version, hardware assumptions, model version, package lockfile, and deployment steps.
Data	Dataset names, licenses, preprocessing, splits, synthetic-data generator or seed, and retention policy.
Runs	Command or UI path, configuration, task IDs, random seeds where relevant, and expected output format.
Costs	API and infrastructure cost estimate, rate limits, retries, and any judge-only stubs or mocks.
Limitations	Known failure modes, unsupported cases, and a concrete plan for reducing risk or false confidence.

Review note Submissions that conceal model versions, use inaccessible private data, or report only selected successes may be downgraded even when the demo appears strong.

Safety boundaries

These boundaries apply to every track. They are part of the problem, not paperwork around it. A project that cannot show how it prevents harmful or unauthorized actions is incomplete.

Boundary	Required interpretation
Authorization	Operate only on systems, accounts, datasets, wallets, devices, and content that the team owns or has explicit permission to use.
Privacy	Use synthetic, consented, or properly licensed data. Do not build deanonymization, doxxing, identity-resolution, or mass-surveillance capabilities.
Cybersecurity	Security testing must stay inside closed labs or user-owned targets. No real-world phishing, credential theft, persistence, destructive exploitation, or third-party scanning.
Agent autonomy	Use least privilege, explicit capability scopes, rate limits, approvals for irreversible actions, and emergency stop/rollback controls.
Payments	Use testnets, simulators, faucet-funded wallets, or capped sandbox accounts. Do not hold participant or public funds, and do not ship an uncontrolled signing key.
Model safety	Do not bypass provider safeguards or jailbreak models. Use approved models, deterministic tools, local models, and policy gates for legitimate evaluation.
Evidence	Keep provenance for outputs. Mark generated or inferred content, preserve uncertainty, and never present a model score as proof of guilt, safety, or mission readiness.

Automatic stop conditions

- 339. The system attempts to access a resource outside its declared scope.
- 340. A payment, command, deletion, publication, or account action exceeds a declared budget or approval policy.
- 341. The system cannot establish evidence provenance or detects prompt injection in an external artifact.
- 342. The project would need real personal data, live offensive infrastructure, or uncontrolled private keys to demonstrate its core claim.

Glossary and protocol notes

Term	Working definition for this handbook
Harness	The control system around a model: context construction, tools, memory, planning, policies, verification, and user interface.
Agent	A model-driven system that can select actions, call tools, observe results, and revise its plan toward a goal.
Context engineering	Deliberate selection, organization, compression, and refresh of information supplied to a model.
Memory	Persisted information with provenance, scope, confidence, and invalidation rules; not an undifferentiated transcript.
Facilitator	In x402, an optional service that verifies payment payloads and settles payments on behalf of a resource server without taking custody of funds.
x402	An HTTP-native payment protocol in which a resource server can return HTTP 402 with payment requirements, receive a payment payload, verify/settle it locally or through a facilitator, and then serve the resource.
Exact scheme	A fixed-price x402 payment pattern suited to one request or a known amount.
Upto scheme	A usage-based x402 pattern in which the client authorizes payment up to a declared limit and the server settles the actual amount used.
Composite/orchestrator	A higher-level system that coordinates multiple x402 resources, providers, agents, or settlement steps rather than serving one paid resource.

x402 protocol notes for Tracks 4 and 5

The x402 challenges assume the current HTTP flow: a client requests a resource; the resource server returns HTTP 402 with payment requirements; the client creates a payment payload; the server verifies it locally or through a facilitator; and the server settles before returning the resource. Participants may implement a compatible subset, but they must document which scheme, network, facilitator, and failure semantics they support.

For participant safety, all payment demonstrations should use a sandbox, testnet, capped wallet, or simulated ledger. The infrastructure entries intentionally emphasize policy, replay protection, duplicate settlement handling, receipts, routing, and observability rather than custody of real funds.

Protocol reference: docs.x402.org — introduction, HTTP 402, client/server, facilitator, wallets, payment schemes, and extensions.

Final reminder The best submission is a system a judge can understand, run, challenge, and trust within the limits you have explicitly declared.